Java Factory pattern

What the Factory pattern is, the three variants (Simple Factory, Factory Method, Abstract Factory), how they differ, and when each fits. The tone is how I would talk each one through with a teammate; the design and experience questions are fully spoken.

1. What is the Factory pattern, why does it exist, and what does it solve?

So the Factory pattern is about **letting a separate piece of code decide which object to create**, instead of the caller doing `new SomeConcreteClass()` directly. It solves **coupling to concrete classes**: the moment you write `new StripeGateway()`, that code is welded to Stripe, and to add PayPal you have to go edit it. A factory hides the "which class" decision behind a method, so callers ask for "a payment gateway" and get one, without knowing or caring about the concrete type. And yes, it is a **creational design pattern** (it is about how objects get made).

2. Why not just use `new` everywhere?

Because `new` **hard-codes the concrete class** at the call site, and that has real costs: the caller is coupled to that exact type, you cannot swap the implementation or mock it in tests, and any creation logic (picking a subtype, wiring dependencies) gets copy-pasted everywhere you construct one. `new` is perfectly fine for simple, stable objects, but when the type varies or creation is non-trivial, a factory centralises that decision in one place. So it is not "never use `new`", it is "do not scatter a varying creation decision across the codebase".

3. What are the advantages and disadvantages?

Advantages: it **decouples callers from concrete classes** (they depend on an interface), centralises creation logic in one spot, makes swapping implementations and mocking easy, and supports the Open/Closed Principle in the Factory Method form. **Disadvantages**: it adds **more classes and indirection**, which is overkill for simple objects, and a badly-done Simple Factory becomes a **giant switch statement** that you keep editing. So the trade is flexibility for a bit more structure.

4. When should you use it, when avoid it, and a real-world example?

Use it when the concrete type **varies** (by config, input, or platform), when creation is **complex**, or when you want callers to depend on an abstraction for testability. **Avoid it** when a plain constructor does the job, forcing a factory onto a simple, single-implementation class is just noise. A **real-world example** everyone has used: `Calendar.getInstance()` hands you the right `Calendar` subclass for your locale without you knowing which, and JDBC's `DriverManager.getConnection()` gives you the right driver's connection. You asked for the abstraction, the factory picked the concrete.

5. What are the three Factory variants, and how do they differ?

There are three, and people mix them up. Quick map:

Variant	What it does	Official GoF?
Simple Factory	One method (often static) returns a product based on a parameter	No, it is an idiom
Factory Method	Subclasses decide which product to create (uses inheritance)	Yes
Abstract Factory	Creates whole **families** of related products (uses composition)	Yes

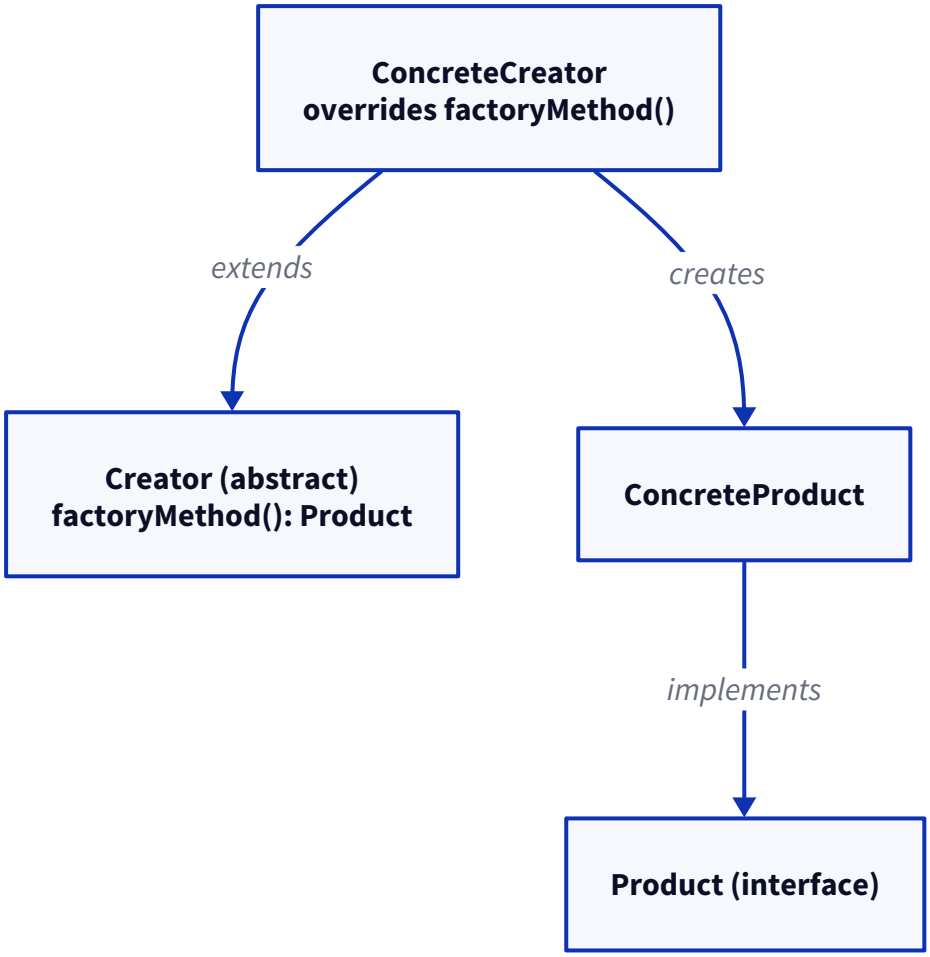
The progression is roughly: **Simple Factory** is a `switch` in one method; **Factory Method** pushes the choice down to subclasses so you extend by adding a subclass; **Abstract Factory** groups several related factory methods so you produce a matching **set** of objects. On the common questions: **Simple Factory is the most used** day to day (it is the simplest), **Factory Method is the easiest to extend** cleanly, and **Abstract Factory suits enterprise cases** where you swap a whole family at once. You **can combine them** (an Abstract Factory's methods are often Factory Methods).

6. Simple Factory: how it works, why it is not official GoF, and does it break OCP?

A Simple Factory is just **one class with a method that returns the right product based on an argument**, usually a `switch` or `if`: `create(type)` returns a `CreditCardPayment`, `PayPalPayment`, and so on. It is **not an official GoF pattern**, it is a common idiom, because it is really just "move the `new` and the `switch` into one method." Its **advantage** is simplicity and one central place for creation; its **disadvantage**, and the classic interview point, is that it **violates the Open/Closed Principle**: every time you add a new product type you have to **open and edit that switch**, risking the existing branches. So you extend it by editing the factory (the OCP smell), and you avoid it once the number of types is growing and changing often, that is the signal to move up to Factory Method.

7. Factory Method: structure, how it supports OCP, and why creation goes to subclasses.

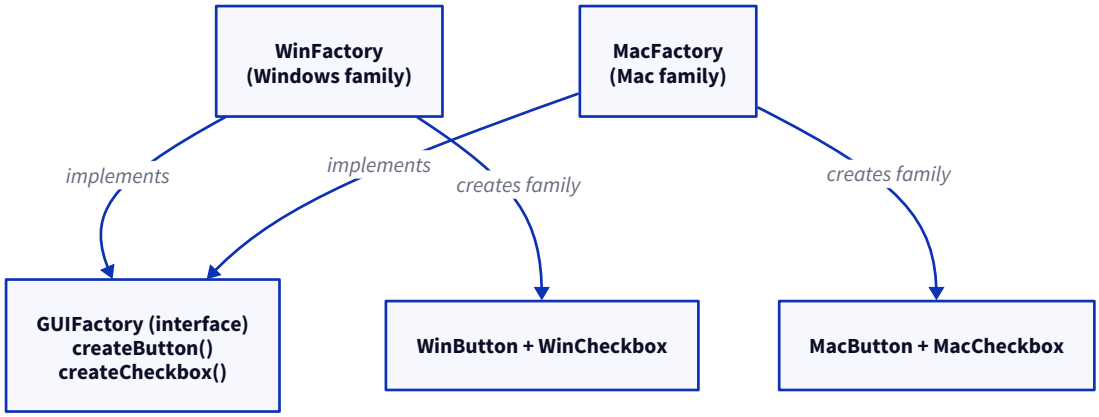
The Factory Method pattern **defines a method for creating a product but lets subclasses decide which concrete product to make**. The structure has two pairs: a **Creator** (abstract, declares the factory method) and its **ConcreteCreator** (overrides it to return a specific product), plus a **Product** interface and its **ConcreteProduct**.



Creation is **delegated to subclasses** so that the base Creator can contain all the shared logic and only the "which product" decision varies per subclass. This is what makes it **support OCP**: to add a new product you write a **new ConcreteCreator**, you do not touch the existing ones, so the code is open to extension but closed to modification. The **downside** is more classes (a creator per product), which is the price of that extensibility. `Collection.iterator()` is a real example: each collection is a creator that returns its own `Iterator`.

8. Abstract Factory: how it creates families, and how it differs from Factory Method.

Abstract Factory is **an interface for creating a family of related objects without naming their concrete classes**. The classic example is a cross-platform UI: a `GUIFactory` with `createButton()` and `createCheckbox()`, and a `WinFactory` that makes Windows-style widgets while a `MacFactory` makes Mac-style ones, so you pick the factory once and get a **matching set**.



The difference from **Factory Method**: Factory Method uses **inheritance** and produces **one** product (subclass decides which); Abstract Factory uses **composition** and produces a **whole family** of related products that are meant to go together. So the tell is "one product v/s a coordinated set." The **downside** is that it is heavy: lots of interfaces and classes, and it is **overkill** when you only have one product type or the families never really vary, that is when Factory Method (or even Simple Factory) is the right size.

9. How does Spring use the Factory pattern, and what is a FactoryBean?

Spring is basically a giant factory. The `BeanFactory` (and its richer `ApplicationContext`) is the **IoC container itself**, the factory that creates and wires all your beans, so you ask the container for a bean instead of `new`-ing it. A `FactoryBean` is different: it is a **special bean that itself acts as a factory for another object**, you implement `getObject()`, and when someone asks for that bean, Spring calls your `getObject()` and hands back what it produces (useful when construction is complex). So the quick distinction: `BeanFactory` **is the container**, and a `FactoryBean` **is one bean whose job is to produce another object**. The names are confusingly similar but they are not the same thing.

10. You are building a payment gateway supporting multiple providers. Factory Method or Abstract Factory?

The way I would decide it comes down to **one product or a family**. If each provider gives me just **one thing**, a payment gateway that I call `charge()` on, then I only need **Factory Method** (or honestly a Simple Factory keyed on provider), because I am picking one concrete `PaymentGateway` per provider. I would reach for **Abstract Factory** only if each provider came with a **coordinated family** of objects that must match, say a gateway plus its own request validator plus its own response parser, and mixing a Stripe gateway with a PayPal parser would be wrong. Then the abstract factory guarantees you get a consistent set. So for a plain "pick the provider's gateway", Factory Method; for "pick the provider's whole toolkit", Abstract Factory. *(Adapt to your real design.)*

11. You are designing a notification system (Email, SMS, Push). How would you apply the Factory pattern?

I would define one `Notifier` **interface** with `send(message)`, and concrete `EmailNotifier`, `SmsNotifier`, `PushNotifier`. Then a **factory returns the right** `Notifier` **for a given channel**, so the calling code just

says "send this over SMS" and never touches the concrete classes. For this shape a **Simple Factory or Factory Method** is plenty, it is one product type (a notifier), chosen by channel. If I wanted it clean and OCP-friendly (adding WhatsApp without editing a switch), I would register the notifiers in a map keyed by channel, so a new channel is a new entry, not a code change. In a Spring app I would let it inject the set of notifiers and pick by type, which is the framework doing the factory work for me. *(Adapt to your own design.)*

12. How would you test a factory implementation?

Two things I would check. First, that the **factory returns the right concrete type for each input** (ask for SMS, assert you get an `SmsNotifier`; ask for an unknown type, assert it throws or returns a sensible default). Second, and more important, that the **factory lets me inject fakes downstream**: because callers depend on the `Notifier` interface rather than a concrete class, I can test their logic with a mock notifier. So the factory itself gets a simple "right type for right input" test, and its real value shows up in how easily everything that depends on it can be tested. *(Adapt to your own setup.)*

13. Does the Factory pattern improve runtime performance or add overhead?

Neither, really, and it is worth being clear about this: the Factory pattern is a **design tool, not a performance tool**. It does **not make anything faster at runtime**, and the **overhead is negligible**, one extra method call and maybe one extra object compared to a direct `new`. So you never choose it for performance; you choose it for **decoupling, flexibility, and testability**. If someone reaches for a factory hoping for speed, they have the wrong reason.

14. Common mistakes.

- **Creating a factory for every class**, adding indirection where a plain constructor was fine.
- **Overusing the pattern**, so simple object creation gets buried under layers.
- **Returning concrete implementations** instead of the abstraction, which throws away the decoupling.
- **Putting business logic inside factories**, when a factory should only create objects.
- **Violating OCP in a Simple Factory** with a switch you keep editing (move to Factory Method or a registry).
- **Letting a factory grow into a giant conditional** with dozens of branches.
- **Mixing Factory and Builder responsibilities** (a factory picks a type; a builder assembles a complex object step by step).
- **Ignoring dependency injection** in modern frameworks, where the container is already the factory.
- **Tight coupling between the factory and concrete implementations**, defeating the point.
- **Using a factory where a constructor is enough**, the most common overengineering.